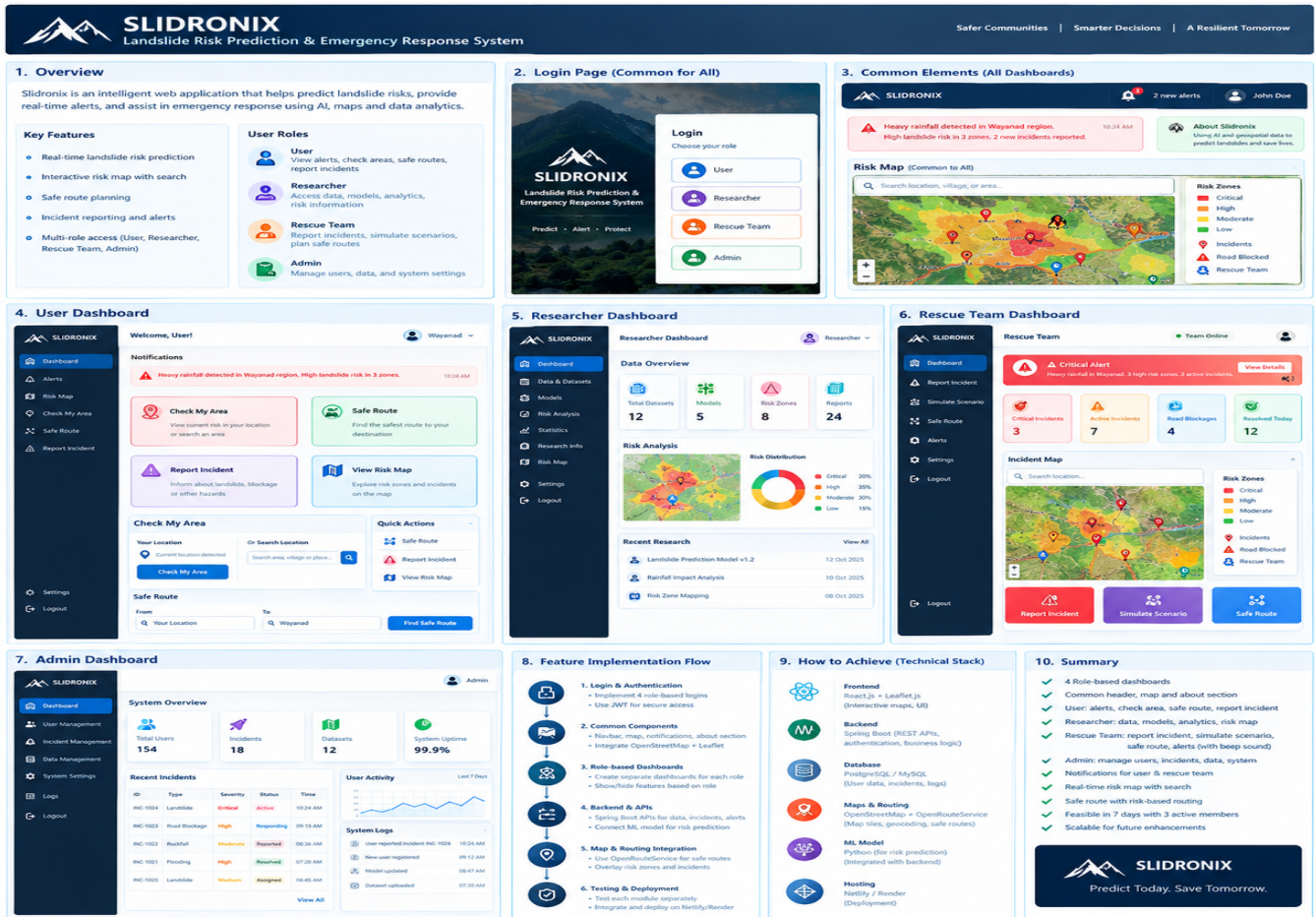


SLIDRONIX

Role-Based Dashboard & Implementation Plan



AI-Based Landslide Early Warning & Emergency Response System

Predict Today. Save Tomorrow.

Project goal: One application with four controlled roles, a common risk map, location-based risk assessment, safe routing, incident reporting, research analytics, and administrative control.

1. Overall System Structure

The system should be one web application, not four separate websites. Login determines the role, and the application loads the appropriate dashboard and permissions.

Role	Main purpose	Key features
User / Citizen	Check personal/local risk and get guidance.	Alerts, common risk map, Check My Area, Safe Route, Report Incident.
Researcher	Study data, models and risk patterns.	Datasets, model information, risk analysis, statistics, research information, risk map.
Rescue Team	Monitor hazards and support emergency response.	Alerts, incident map, Report Incident, Simulate Scenario, Safe Route.
Admin	Operate and control the whole platform.	Users, incidents, alerts, datasets, models, system analytics and settings.

Common foundation

- Login page with four roles: User, Researcher, Rescue Team and Admin.
- SLIDRONIX title/logo and a short About section available across the application.
- Interactive Risk Map with a location search bar as a shared core feature.
- Role-based permissions so each role sees only its intended functions.
- Backend APIs and database shared by all dashboards.

2. Common Features & Navigation

Feature	Who sees it?	How it works
SLIDRONIX title + short About	All four	Same branding and a concise explanation of the system.
Risk Map + search	All four	Leaflet map displays risk zones, searchable locations and relevant markers.
Emergency notification	User + Rescue Team	Active alert appears near the top; critical rescue alerts can also play a beep.
Login / role selection	All	Authentication identifies the role and loads the correct dashboard.
Logout / profile	All	Secure session controls and basic account information.

- Notification rule: Users and rescue teams receive emergency incident/risk notifications. Researchers do not need the emergency notification stream; they can access the information through research/risk analysis.
- Incident propagation: A user or rescue team submits an incident → backend stores it → admin can manage it → user/rescue dashboards receive the relevant alert.
- Important: The system should describe ML output as a risk estimate, not a guaranteed landslide prediction.

3. User / Citizen Dashboard

Purpose: Give a normal user a simple way to understand the risk around them, receive important warnings, check a specific area, plan a safer route and report an observed hazard.

A. Emergency notification

- Show active warnings near the top when there is a relevant high/critical event.
- Example: Heavy rainfall detected; high landslide risk in selected zones.
- Optional sound alert for critical incidents; provide mute control to avoid repeated noise.

B. Common risk map

- Search village, road, city or area.
- Display color-coded risk zones and relevant incident markers.
- Clicking a zone can show risk level, score and basic explanation.

C. Check My Area

- Ask for browser location permission after login or when the feature is first used.
- Also provide manual location search for users who do not allow location access.
- Send location and available environmental inputs to the backend/ML service.
- Return current estimated risk, risk level, key contributing factors and basic advice.

D. Safe Route

- From = current location or searched location; To = searched destination.
- Routing service returns route geometry and alternatives.
- Slidronix compares route paths with its risk zones and ranks them by hazard exposure.
- Show safest/recommended route plus alternatives with distance, time and risk level.

E. Report Incident

- Allow citizens to report landslide, rockfall, road blockage, flooding, fallen tree, structural damage or other hazards.
- Capture location, incident type, severity and description; photo can be added later.
- Submit to backend and trigger relevant alerts for users/rescue teams.

User dashboard flow: Login → allow/search location → view alert/map → Check My Area → Safe Route → optionally Report Incident.

4. Researcher Dashboard

Purpose: Provide a controlled research workspace without exposing emergency-response controls.

Module	What it contains	Achievable implementation
Data & Datasets	Dataset overview, feature names, sample counts, source information and basic filters.	Backend endpoint reads dataset metadata; files can remain on the server/project storage.
Models	Random Forest and other models used by Slidronix; model version, input features and basic performance metrics.	Store model metadata and expose it through a Spring Boot API; Python ML service can serve predictions.
Risk Analysis	Risk distribution, high-risk regions, contributing factors and trend views.	Use backend-processed aggregates and Chart.js visualizations.
Statistics	Counts, percentages, risk classes, rainfall/risk relationships and historical summaries.	Generate SQL/Python aggregates and return JSON for charts.
Research Information	Project methodology, literature summary, features, limitations and future work.	Static React pages/Markdown content; no complex backend required.
Risk Map	Same shared map with research-focused layers.	Leaflet + GeoJSON risk zones and searchable locations.

Keep the researcher role limited

- No emergency beep/notification stream is required.
- No user-account management or rescue dispatch controls.
- Focus on datasets, models, statistics, analysis and research information.

5. Rescue Team Dashboard

Purpose: Turn Slidronix into an operational aid for monitoring hazards, receiving critical alerts, reporting incidents, assessing scenarios and planning safer response routes.

Module	What the rescue team can do	Result
Emergency Alerts	See high/critical alerts at the top; optional beep for critical events.	Faster awareness of urgent conditions.
Risk / Incident Map	View risk zones, incident markers, road-blockage markers and search locations.	One visual operating picture.
Report Incident	Location + type + severity + description; submit incident.	Incident is stored and relevant users/rescue teams are alerted.
Simulate Scenario	Choose/check a location and run the current scenario through the risk model.	Estimated risk level, score, factors and recommended monitoring/action.
Safe Route	Set incident/rescue origin and destination.	Routes ranked using Slidronix risk-zone exposure plus travel information.
Incident Status	View reported/responding/resolved incidents.	Shared operational status for the team.

Incident alert flow

- User/Rescue Team submits incident.
- Backend validates and stores incident.
- Admin can review/manage it.
- Relevant User + Rescue Team dashboards receive notification.
- Critical severity can trigger a short beep on Rescue Team dashboard.
- Incident appears on the shared map with its severity/status.

6. Admin Dashboard

Purpose: Give the administrator complete system-level visibility and control while keeping the user/rescue/researcher interfaces focused.

Area	Admin functions
System Overview	Total users, active incidents, resolved incidents, datasets, models and system status.
User Management	View users, roles, activate/deactivate accounts and basic role administration.
Incident Management	Review incidents, verify/update severity, assign status, resolve/close incidents.
Alert Management	Create/manage announcements and emergency alert rules/messages.
Data Management	Manage dataset metadata and model versions; upload/revise research assets if needed.
Analytics / Logs	Basic activity, incident trends, login/activity logs and system events.
Settings	Risk thresholds, notification settings and other application configuration.

Admin should not become a second research dashboard

- Give admin operational controls, not every technical research detail.
- Keep sensitive management actions behind admin permissions.
- Use logs so important actions such as changing incident status can be traced.

7. How the Features Can Be Achieved

Requirement	Suggested technology	Implementation idea
UI + role dashboards	React.js	React Router + protected routes; render dashboard components based on authenticated role.
Interactive maps	Leaflet + OpenStreetMap	React-Leaflet displays tiles, markers, GeoJSON risk zones and route polylines.
Location	Browser Geolocation API	Ask permission and obtain latitude/longitude; provide manual search as fallback.
Location search / geocoding	Pelias / current openrouteservice ecosystem	Convert a typed place name into coordinates.
Routing	openrouteservice / current HeiGIT API	Request directions/alternative routes; draw returned route geometry on Leaflet.
Risk-aware route	Slidronix risk GeoJSON + route geometry	Score how much each route overlaps/buffers through risk zones and rank safest vs alternatives.
Backend	Spring Boot	REST APIs for authentication, risk checks, incidents, alerts, research data and admin operations.
Database	PostgreSQL / MySQL	Users, roles, incidents, alerts, locations, statuses, logs and metadata.
ML	Python + existing model	Expose prediction through a Python service or integrate the model behind the backend.
Charts	Chart.js	Research statistics, risk distribution and admin summaries.
Authentication	JWT + Spring Security	Secure login and role-based API authorization.

8. Core Data Flows

Check My Area:

User → location permission/manual search → latitude/longitude → backend → environmental/available feature values → ML model → risk score/class → dashboard explanation + map.

Safe Route:

From/To → geocoding → coordinates → routing API → route alternatives → Slidronix risk-zone intersection/exposure scoring → safest route + alternatives → Leaflet map.

Incident Reporting:

User/Rescue → incident form → backend validation → database → alert service → User + Rescue notifications → map marker → Admin review/status.

Simulate Scenario:

Rescue → selected/current location → scenario/environmental inputs → ML prediction → risk score/class + factors → rescue dashboard.

Research:

Researcher → dataset/model/risk-analysis API → processed metadata/statistics → charts/tables/map layers.

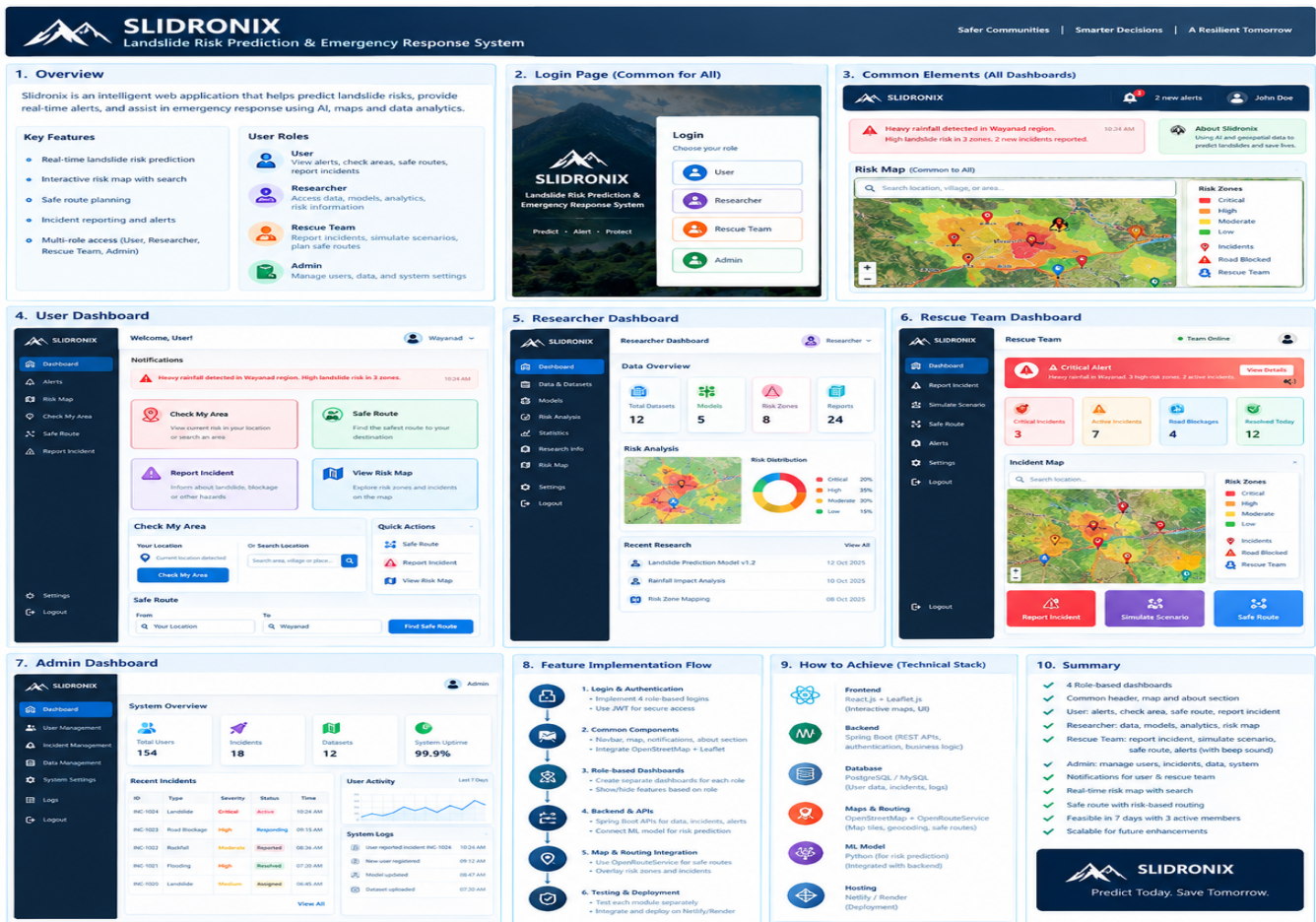
9. Recommended Database Entities

Keep the database simple for the first version. A small number of clear tables is easier to integrate and demonstrate.

Entity	Important fields
users	id, name, email, password_hash, role, status, created_at
incidents	id, reporter_id, latitude, longitude, type, severity, description, status, created_at
alerts	id, title, message, severity, target_role, active, created_at
risk_zones	id, geometry/GeoJSON reference, risk_level, risk_score, region
model_versions	id, model_name, version, features, metrics, active
datasets	id, name, source, records, features, version, description
logs	id, user_id, action, timestamp, metadata

10. UI Design Direction

The generated concept below is a design reference, not a final screenshot. The actual React implementation can use the same hierarchy and components.



Recommended visual language: dark blue/navy Slidronix header, white cards, clean map area, and consistent severity colors: red = critical, orange = high, yellow = moderate, green = low. Keep emergency information visually dominant but avoid clutter.

11. Practical Build Order for Your Team

Day	Build target
1	Project structure, login, four roles, protected routes, common header/about/map shell.
2	User dashboard: alerts, common map/search, Check My Area UI and location/manual search.
3	ML/backend integration for Check My Area + risk result display.
4	Safe Route: From/To, geocoding, route display, alternatives and basic risk ranking.
5	Rescue dashboard: Report Incident, incident map, notifications and Simulate Scenario.
6	Researcher + Admin dashboards; connect database, incident management and statistics.
7	Integration, testing, UI polish, permissions, notification/beep behavior, demo data and presentation.

If time becomes tight

- Do not build live IoT sensors, complex physics simulation, a custom routing engine, real emergency dispatch, or full live satellite processing.
- Prioritize login/roles, risk map, user Check My Area, one working ML prediction path, incident reporting, rescue alerts and a demonstrable Safe Route.
- Use demo/static risk GeoJSON where necessary so the application can be completed and demonstrated reliably.

12. Final Dashboard Checklist

- ✓ One login with four roles: User, Researcher, Rescue Team, Admin.
- ✓ Common SLIDRONIX branding, short About section and searchable Risk Map.
- ✓ Emergency notifications for User + Rescue Team; optional critical beep for Rescue Team.
- ✓ User: Check My Area + Safe Route + Report Incident.
- ✓ Researcher: Data, Models, Risk Analysis, Statistics, Research Information + Risk Map.
- ✓ Rescue Team: Alerts, Risk/Incident Map, Report Incident, Simulate Scenario + Safe Route.
- ✓ Admin: Users, Incidents, Alerts, Data, Models, Analytics, Logs and Settings.
- ✓ Incident reports become shared system events and map markers.
- ✓ Safe Route is risk-aware: shortest is not automatically safest.
- ✓ All dashboards are connected through one backend, database and ML/risk service.

SLIDRONIX
Predict Today. Save Tomorrow.